

LAB 3. ENVIRONMENTAL SOUND CLASSIFICATION

URBANSOUND8K WITH 1D-CNN

ThS. Lê Thị Thùy Trang

2026-04-27

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau bài lab này, sinh viên sẽ chuyển từ bài toán phân loại ảnh bằng CNN ở Lab 2 sang bài toán phân loại âm thanh môi trường bằng **1D-CNN**. Dữ liệu âm thanh không còn là ma trận pixel như ảnh, mà là tín hiệu thay đổi theo thời gian. Vì vậy, mục tiêu chính của buổi học không phải là train mô hình thật to cho oách, mà là hiểu cách đưa audio về một biểu diễn phù hợp, rồi dùng convolution một chiều để học các mẫu cục bộ theo thời gian.

Trong bài lab này, cấu hình chính được chọn là **MFCC + 1D-CNN** để lab chạy ổn trên laptop cá nhân. Cấu hình **1D-CNN trực tiếp trên raw waveform** đã được chuẩn bị trong repo, nhưng được đặt là bài mở rộng cho sinh viên muốn thử nghiệm sâu hơn.

- Hiểu bài toán phân loại âm thanh môi trường trên bộ dữ liệu UrbanSound8K;
- Biết cách chuẩn hóa audio về cùng sample rate, cùng số kênh và cùng độ dài;
- Trích xuất được đặc trưng MFCC hoặc log-mel theo thời gian;
- Xây dựng và huấn luyện mô hình 1D-CNN cho chuỗi đặc trưng audio;
- Dùng `Weights & Biases` để log quá trình huấn luyện;
- Đọc learning curves và phát hiện overfitting/underfitting;
- Phân tích confusion matrix để biết mô hình hay nhầm lớp nào;
- Rút ra kết luận dựa trên số liệu thay vì cảm giác.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được với bài toán phân loại âm thanh bằng 1D-CNN;
2. Một pipeline tiền xử lý audio chạy được trên tập UrbanSound8K;
3. Một mô hình MFCC + 1D-CNN chạy ổn và có log trên W&B;
4. Ít nhất 1 run chính cho baseline MFCC + 1D-CNN;
5. Learning curves cho train và validation;
6. Confusion matrix cho kết quả test;
7. Một bảng tổng hợp metrics và thời gian train/epoch;

8. Phân tích ngắn: vì sao dùng MFCC/log-mel sẽ ổn định hơn raw waveform trong buổi lab này.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy.
3. Link hoặc ảnh chụp W&B dashboard.
4. Kết quả của mô hình baseline MFCC + 1D-CNN.
5. File `curves.png`.
6. File `confusion_matrix.png`.
7. File `metrics.json` hoặc bảng tổng hợp metrics.
8. Kết quả test của best model.
9. Phân tích ngắn: lớp nào dễ bị nhầm, vì sao có thể bị nhầm.
10. Bài mở rộng nếu có: so sánh thêm log-mel + 1D-CNN hoặc raw waveform + 1D-CNN.

1.4 Bối cảnh bài toán

Chúng ta chuyển sang đóng vai nhóm kỹ sư AI hỗ trợ xây dựng hệ thống nhận diện âm thanh môi trường trong đô thị. Mỗi file đầu vào là một đoạn âm thanh ngắn, ví dụ tiếng còi xe, tiếng chó sủa, tiếng khoan, tiếng điều hòa, tiếng trẻ em chơi hoặc tiếng nhạc đường phố. Mô hình cần dự đoán đoạn âm thanh đó thuộc loại nào.

Bộ dữ liệu sử dụng là **UrbanSound8K**, sinh viên có thể truy cập đường dẫn [này](#) để lấy dữ liệu. Tập dữ liệu gồm 10 lớp âm thanh môi trường:

- air_conditioner
- car_horn
- children_playing
- dog_bark
- drilling
- engine_idling
- gun_shot
- jackhammer
- siren
- street_music

Nếu ở Lab 2 chúng ta để CNN nhìn ảnh theo cấu trúc không gian, thì ở Lab 3 chúng ta để 1D-CNN nhìn chuỗi đặc trưng audio theo thời gian. Điểm giống nhau là convolution vẫn học các mẫu cục bộ. Điểm khác nhau là bây giờ các mẫu cục bộ nằm trên trục thời gian của âm thanh.

1.5 Cấu trúc repo starter kit

Repo của bài lab có cấu trúc như sau:

```
csc4005_lab3_urbansound8k_1dcnn_starter/  
  README.md  
  REPORT_TEMPLATE.md  
  requirements.txt  
  configs/  
    baseline_mfcc_1dcnn.json  
    fast_debug.json  
    extension_raw_waveform.json  
  docs/  
    WANDB_GUIDE.md  
  |  RUBRIC.md  
    LAB_GUIDE_LAB3.md  
  notebooks/  
    lab3_demo.ipynb  
  outputs/  
  src/  
    __init__.py  
    dataset.py  
    model.py  
    train.py  
    utils.py  
  ci/  
    check_structure.py  
    smoke_train.py  
  .github/  
    workflows/  
      validate-lab3.yml
```

Ý nghĩa từng phần

- `src/dataset.py`: đọc metadata UrbanSound8K.csv, đọc file audio, resample, pad/crop và trích xuất đặc trưng.
- `src/model.py`: định nghĩa mô hình MFCC/log-mel 1D-CNN và mô hình mở rộng raw waveform 1D-CNN.
- `src/train.py`: huấn luyện mô hình, đánh giá, lưu output, log W&B.
- `src/utils.py`: các hàm tiện ích như vẽ curves, confusion matrix, lưu metrics.
- `configs/baseline_mfcc_1dcnn.json`: cấu hình chính khuyến nghị cho buổi lab.
- `configs/fast_debug.json`: cấu hình chạy nhanh để kiểm tra pipeline.
- `configs/extension_raw_waveform.json`: cấu hình mở rộng cho raw waveform.
- `docs/WANDB_GUIDE.md`: hướng dẫn riêng cho W&B.
- `REPORT_TEMPLATE.md`: mẫu báo cáo cho sinh viên.

2 Chuẩn bị môi trường thực hành

2.1 Điều kiện cần

Sinh viên cần:

- Có tài khoản GitHub và đăng nhập được.
- Máy có Internet.
- Môi trường ảo `csc4005-dl` đã được set-up từ các buổi trước.
- Đã biết cách dùng `wandb login`.
- Đã tải được bộ dữ liệu UrbanSound8K.

2.2 Clone repo

Bấm **Accept Assignment** theo link đã cung cấp trên Notion hoặc GitHub Classroom.

Clone repo về máy

```
git clone <repo-url>
cd <ten-repo>
```

2.3 Khởi tạo environment

```
conda activate csc4005-dl
```

Nếu chưa có environment từ các buổi trước thì có thể tạo mới:

```
conda create -n csc4005-dl python=3.10 -y
conda activate csc4005-dl
```

2.4 Cài thư viện phục vụ bài lab

Thực tế thì mình đã cài khá nhiều thư viện từ các bài lab trước rồi, nhưng vẫn để file `requirements.txt` trong repo và hướng dẫn ở đây cho nó chắc chắn nhé.

```
pip install -r requirements.txt
```

Bài này có xử lý audio nên sẽ dùng thêm `librosa`, `soundfile` và một số thư viện liên quan. Nếu máy báo lỗi khi cài `soundfile` hoặc đọc file audio, hãy kiểm tra lại môi trường Python trước khi đổ lỗi cho mô hình nhé.

2.5 Chuẩn bị dữ liệu UrbanSound8K

Sau khi tải và giải nén, thư mục dữ liệu nên có dạng:

```
UrbanSound8K/
  audio/
    fold1/
    fold2/
    fold3/
    ...
  metadata/
    UrbanSound8K.csv
```

Khi chạy, sinh viên truyền đường dẫn tới thư mục này qua tham số `--data_dir`.

Ví dụ:

```
python -m src.train \
--config configs/fast_debug.json \
--data_dir /duong_dan/UrbanSound8K
```

Nếu dùng Windows, đường dẫn có thể giống như sau:

```
python -m src.train \
--config configs/fast_debug.json \
--data_dir "D:/datasets/UrbanSound8K"
```

Lưu ý nhỏ nhưng quan trọng: repo không chứa dữ liệu. Không push thư mục data/, file .wav, file zip lớn hoặc cache feature lên GitHub.

2.6 Nhắc lại thao tác với W&B

Nếu máy đã đăng nhập từ buổi trước thì có thể bỏ qua. Nếu chưa, thực hiện lại:

```
wandb login
```

Dán API key khi được yêu cầu.

Trong lab này, W&B là yêu cầu bắt buộc. Nếu mạng yếu, có thể chạy offline trước:

```
python -m src.train \
--config configs/baseline_mfcc_1dcnn.json \
--data_dir /duong_dan/UrbanSound8K \
--wandb_mode offline
```

Sau đó đồng bộ lại:

```
wandb sync wandb/offline-run-...
```

3 Voila, hướng dẫn thực hành ở đây

3.1 Kiểm tra pipeline bằng fast debug

Đừng vội chạy baseline chính ngay. Với audio, lỗi đường dẫn, lỗi đọc file, lỗi thiếu metadata, lỗi thư viện âm thanh rất dễ xảy ra. Vì vậy, ta chạy debug trước cho yên tâm.

```
python -m src.train \
--config configs/fast_debug.json \
--data_dir /duong_dan/UrbanSound8K
```

Run debug giúp kiểm tra:

- code có tìm được UrbanSound8K.csv không;
- code có tìm được file .wav trong các thư mục fold1, fold2, ... không;
- audio có đọc được bằng librosa không;
- feature cache có hoạt động không;
- model có train được ít nhất vài epoch không;
- W&B có log được không.

Nếu debug đã chạy ổn thì mới sang baseline chính. Cứ đi từ nhỏ đến lớn, đỡ đau tim hơn.

3.2 Chạy baseline chính: MFCC + 1D-CNN

Tinh thần của bài lab này là không nhảy ngay vào raw waveform cho thật deep, mà phải bắt đầu bằng một cấu hình ổn định để hiểu chính xác 1D-CNN đang học cái gì.

Ta dùng baseline chính:

```
python -m src.train \
  --config configs/baseline_mfcc_1dcnn.json \
  --data_dir /duong_dan/UrbanSound8K
```

Nếu muốn đặt tên run theo mã sinh viên:

```
python -m src.train \
  --config configs/baseline_mfcc_1dcnn.json \
  --data_dir /duong_dan/UrbanSound8K \
  --run_name mssv_mfcc_1dcnn_baseline
```

Cấu hình baseline được thiết kế để lab chạy ổn:

- `sample_rate = 16000`: đưa audio về cùng tần số lấy mẫu;
- `duration = 4.0`: đưa clip về cùng độ dài 4 giây;
- `feature_type = mfcc`: dùng MFCC làm chuỗi đặc trưng;
- `n_mfcc = 40`: mỗi frame có 40 hệ số MFCC;
- `train_folds = 1..8`: dùng fold 1 đến fold 8 để train;
- `val_folds = 9`: dùng fold 9 để validation;
- `test_folds = 10`: dùng fold 10 để test;
- `max_train_per_class = 120`: giới hạn số mẫu mỗi lớp để chạy ổn trong lớp;
- `max_eval_per_class = 50`: giới hạn số mẫu validation/test mỗi lớp;
- `use_wandb = true`: bật logging lên W&B.

Nhiều tham số quá nhi, chịu khó đọc giải thích tham số nha.

- `--config`: đường dẫn file cấu hình JSON. Trong lab này nên dùng file config để tránh gõ quá nhiều tham số.
- `--data_dir`: **bắt buộc**, là đường dẫn tới thư mục UrbanSound8K hoặc file zip.
- `--project`: tên project trên W&B. Trong lab này dùng thống nhất là `csc4005-lab3-urbansound-1dcnn`.
- `--run_name`: tên run.
- `--model_name`: tên mô hình. Với baseline đầu tiên dùng `mfcc_1dcnn`.
- `--feature_type`: kiểu đặc trưng đầu vào, có thể là `mfcc`, `logmel`, hoặc `raw`.
- `--sample_rate`: sample rate sau khi resample.
- `--duration`: độ dài audio sau khi pad/crop.
- `--n_mfcc`: số hệ số MFCC.
- `--n_mels`: số mel bands nếu dùng log-mel.
- `--hidden_channels`: số kênh ẩn trong các block Conv1D.
- `--optimizer`: chọn optimizer, ví dụ `adamw` hoặc `sgd`.
- `--scheduler`: có thể là `none` hoặc `plateau`.

- `--lr`: learning rate.
- `--weight_decay`: hệ số regularization L2.
- `--dropout`: hệ số dropout.
- `--epochs`: số vòng lặp huấn luyện.
- `--batch_size`: số mẫu trong một batch.
- `--patience`: số epoch chờ để early stopping.
- `--augment`: bật augmentation nhẹ cho train set.
- `--cache_dir`: thư mục lưu feature đã trích xuất để lần chạy sau nhanh hơn.
- `--use_wandb`: bật logging lên W&B.
- `--wandb_mode`: chế độ log, thường dùng `online`, khi mạng yếu có thể dùng `offline`.

Chạy baseline xong rồi thì nhớ:

1. Mở dashboard W&B.
2. Kiểm tra train loss và val loss.
3. Kiểm tra train accuracy và val accuracy.
4. Kiểm tra thời gian train/epoch.
5. Mở file `confusion_matrix.png`.
6. Ghi chú lớp nào dễ bị nhầm.
7. Giữ run này làm mốc so sánh với các thử nghiệm mở rộng.

3.3 Vì sao không đưa raw waveform làm cấu hình chính?

Đây là phần quan trọng nhất của buổi học. Nếu chỉ chăm chăm chạy code mà không nắm được ý này thì hơi phí.

Audio gốc ở dạng waveform là một chuỗi rất dài. Ví dụ audio 4 giây với sample rate 16000 Hz sẽ có khoảng 64000 điểm tín hiệu. Nếu đưa trực tiếp waveform vào 1D-CNN, mô hình phải tự học rất nhiều thứ từ đầu: biên độ, nhịp, tần số, pattern lặp lại, nhiễu nền, v.v.

MFCC/log-mel thì khác. Đây là biểu diễn đã tóm tắt thông tin phổ âm thanh theo thời gian. Nói nôm na, thay vì bắt mô hình tự mò từ tín hiệu thô, ta đưa cho mô hình một dạng đầu vào gọn hơn, có ý nghĩa hơn và dễ học hơn trong thời lượng lab.

- Raw waveform: gần dữ liệu gốc hơn nhưng khó học hơn.
- MFCC: gọn, ổn định, phù hợp cho baseline trên laptop.
- Log-mel: giữ nhiều thông tin phổ hơn MFCC, thường là biến thể tốt để thử thêm.

Nói ngắn gọn: trong lab này, ta chọn **MFCC + 1D-CNN** không phải vì raw waveform không hay, mà vì cần một cấu hình đủ ổn định để sinh viên tập trung hiểu đúng quy trình thực nghiệm.

3.4 1D-CNN đang học gì trong bài toán audio?

Với baseline MFCC, input đưa vào model có dạng gần như sau:

```
[batch_size, n_mfcc, time_frames]
```

Trong đó:

- `batch_size`: số mẫu trong một batch;
- `n_mfcc`: số kênh đặc trưng, ví dụ 40;
- `time_frames`: số frame theo thời gian.

Conv1D sẽ trượt dọc theo trục thời gian để học các mẫu cục bộ. Ví dụ:

- tiếng chó sủa có thể có các đoạn năng lượng ngắn và rõ;
- tiếng điều hòa có thể đều và kéo dài;
- tiếng còi xe có thể có pattern cao độ nổi bật;
- tiếng khoan hoặc jackhammer có thể có pattern lặp lại mạnh.

Nếu ở ảnh, CNN học cạnh, góc, texture, thì ở audio, 1D-CNN học các pattern biến thiên theo thời gian.

3.5 Nếu nhìn những biểu đồ như nhìn bức vách thì không sao hết, đọc hướng dẫn dưới đây là ổn áp ngay

Khi training xong, W&B sẽ cho chúng ta các đường cong học tập.

3.5.1 Những đường quan trọng nhất

- `train_loss`
- `val_loss`
- `train_acc`
- `val_acc`
- `epoch_time_sec`

3.5.2 Trường hợp tốt

- `train_loss` giảm đều,
- `val_loss` giảm đều hoặc giảm rồi ổn định,
- `train_acc` tăng,
- `val_acc` tăng theo,
- khoảng cách giữa train và validation không quá lớn.

Đây là dấu hiệu mô hình đang học đúng hướng.

3.5.3 Dấu hiệu overfitting

- `train_acc` tăng rất cao,
- `train_loss` tiếp tục giảm,
- nhưng `val_loss` bắt đầu tăng,
- `val_acc` không tăng tương ứng hoặc giảm.

Lúc này mô hình đang ghi nhớ train set quá mức. Với UrbanSound8K, điều này có thể xảy ra vì dữ liệu âm thanh khá nhiều, số lượng mẫu mỗi lớp không quá lớn và các lớp có thể dễ nhầm.

3.5.4 Dấu hiệu underfitting

- cả train accuracy và val accuracy đều thấp,
- train loss và val loss đều còn cao,
- mô hình gần như đoán lung tung sau nhiều epoch.

Lúc này mô hình chưa học đủ hoặc mô hình quá yếu. Cũng có thể do feature chưa phù hợp, learning rate chưa ổn hoặc dữ liệu đầu vào đang bị đọc sai.

3.6 Đọc confusion matrix

Learning curves cho ta biết mô hình học có ổn không. Confusion matrix cho ta biết mô hình đang nhầm ở đâu.

Sau khi train xong, mở:

```
outputs/<run_name>/confusion_matrix.png
```

Hãy quan sát:

- lớp nào có số dự đoán đúng nhiều nhất;
- lớp nào bị nhầm sang lớp khác nhiều nhất;
- có cặp lớp nào hay nhầm lẫn với nhau không;
- lỗi nhầm đó có hợp lý về mặt âm thanh không.

Ví dụ, một số âm thanh đô thị có thể cùng chứa nhiều nền hoặc có pattern năng lượng gần nhau, nên mô hình nhầm là chuyện bình thường. Điều quan trọng không phải là giấu lỗi đi, mà là nhìn vào lỗi để hiểu mô hình.

3.7 Thử biến thể log-mel + 1D-CNN

Sau khi baseline MFCC đã chạy ổn, sinh viên có thể thử thêm log-mel. Đây vẫn là 1D-CNN trên chuỗi đặc trưng theo thời gian, nhưng đầu vào khác một chút.

```
python -m src.train \
  --config configs/baseline_mfcc_1dcnn.json \
  --data_dir /duong_dan/UrbanSound8K \
  --feature_type logmel \
  --run_name mssv_logmel_1dcnn
```

Khi so sánh với MFCC, đừng chỉ hỏi “accuracy có cao hơn không?”. Hãy hỏi thêm:

- log-mel có train chậm hơn không;
- val loss có ổn định hơn không;
- confusion matrix có giảm nhầm lẫn ở lớp nào không;
- kết quả có lặp lại ổn định nếu đổi seed không.

3.8 Bài mở rộng: 1D-CNN trực tiếp trên raw waveform

Đến phần được trông chờ đây, nhưng nhớ đây là **bài mở rộng**, không phải phần bắt buộc cho tất cả sinh viên trong buổi lab.

Chạy raw waveform như sau:

```
python -m src.train \
--config configs/extension_raw_waveform.json \
--data_dir /duong_dan/UrbanSound8K \
--run_name mssv_raw_waveform_extension
```

Ở cấu hình này, input đưa vào model gần như là:

```
[batch_size, 1, samples]
```

Tức là model nhìn trực tiếp tín hiệu âm thanh gốc. Nghe thì rất ngẫu, nhưng cũng khó hơn. Mô hình phải học từ tín hiệu thô nên có thể:

- train chậm hơn;
- cần batch size nhỏ hơn;
- dễ overfitting hơn;
- cần nhiều dữ liệu hoặc kiến trúc tốt hơn;
- không chắc thắng baseline MFCC trong lab gần.

Lưu ý: Không yêu cầu raw waveform phải tốt hơn MFCC. Nếu raw waveform kém hơn, đó vẫn là một kết quả tốt nếu sinh viên giải thích được nguyên nhân.

3.9 So sánh các run trên W&B

Sau khi chạy ít nhất baseline chính, vào project trên W&B và kiểm tra.

Nếu có nhiều run, hãy so sánh:

- run nào có `best_val_acc` cao nhất,
- run nào có `val_loss` thấp nhất,
- run nào learning curve ổn định hơn,
- run nào train nhanh hơn theo epoch,
- run nào ít overfitting hơn,
- run nào có confusion matrix hợp lý hơn.

Lưu ý: Kết luận phải dựa trên số liệu và dashboard, không nên viết cảm tính. Không viết kiểu “em thấy model này có vẻ tốt hơn”. Hãy viết “model này có best val accuracy cao hơn, val loss ổn định hơn và nhầm ít hơn ở lớp ...”.

3.10 Bước cuối cùng: so bó đũa, chọn cột cờ

3.10.1 Chọn best model

Best model được chọn dựa trên kết quả train model.

Nguyên tắc đúng là:

1. So sánh các cấu hình trên **validation set**.
2. Chọn **một cấu hình tốt nhất**.
3. Ghi rõ lý do chọn.
4. Chỉ sau đó mới dùng **test set** để đánh giá cuối cùng.

Tiêu chí chọn best config có thể là:

- val accuracy cao nhất,
- val loss thấp và ổn định,
- learning curves đẹp,
- ít overfitting hơn,
- thời gian train/epoch hợp lý hơn,
- confusion matrix dễ giải thích hơn.

Hết sức lưu ý: Không được chọn mô hình chỉ vì train accuracy cao.

3.10.2 Đánh giá trên test set

Sau khi chọn được best config, sinh viên đánh giá trên test set **một lần cuối**.

Trong starter kit, quá trình test đã được thực hiện ở cuối mỗi run. Kết quả được lưu trong:

```
outputs/<run_name>/
```

Thông thường sẽ có:

- best_model.pt
- history.csv
- curves.png
- confusion_matrix.png
- metrics.json
- used_config.json

Kết quả nên có:

- test accuracy,
- classification report,
- confusion matrix,
- một vài nhận xét về lớp dễ đúng,
- một vài nhận xét về lớp dễ sai.

4 Khi nào MFCC/log-mel tốt hơn raw waveform?

Đây là câu hỏi quan trọng của lab.

MFCC/log-mel thường phù hợp hơn trong buổi lab khi:

- thời gian thực hành ngắn;
- máy sinh viên không quá mạnh;
- muốn một baseline ổn định;
- muốn giảm số chiều đầu vào;
- muốn mô hình nhỏ vẫn học được pattern hữu ích;

- mục tiêu chính là hiểu pipeline audio classification.

Tuy nhiên, điều này không có nghĩa raw waveform là hướng kém. Raw waveform có thể rất mạnh nếu có kiến trúc tốt, dữ liệu đủ nhiều và thời gian huấn luyện phù hợp. Trong lab này, raw waveform được đề làm bài mở rộng để sinh viên khá/giỏi thấy được sự khác biệt giữa học từ feature đã trích xuất và học trực tiếp từ tín hiệu gốc.

Bởi vậy, kết luận cuối cùng phải dựa trên số liệu của chính các em.

5 Những lỗi rất hay gặp

5.1 Lỗi 1. Sai đường dẫn dữ liệu

Thông báo thường gặp là không tìm thấy UrbanSound8K.csv hoặc không tìm thấy file audio. Hãy kiểm tra lại cấu trúc thư mục audio/ và metadata/.

5.2 Lỗi 2. Push dữ liệu hoặc cache lên GitHub

Không push file .wav, file zip dataset, thư mục data/ hoặc thư mục .cache/ lên GitHub. Repo chỉ nên chứa code, config và tài liệu.

5.3 Lỗi 3. Nhầm lẫn giữa sample và frame

Sample là điểm tín hiệu âm thanh trong waveform. Frame là lát thời gian sau khi trích xuất MFCC/log-mel. Hai khái niệm này không giống nhau.

5.4 Lỗi 4. Hiểu sai shape của input

Với MFCC, input thường có dạng:

```
[batch_size, n_mfcc, time_frames]
```

Với raw waveform, input thường có dạng:

```
[batch_size, 1, samples]
```

Nhầm shape là mô hình lỗi ngay, hoặc tệ hơn là chạy được nhưng học rất kỳ lạ.

5.5 Lỗi 5. Chỉ nhìn accuracy mà không nhìn loss

Accuracy không phải lúc nào cũng kể hết câu chuyện. Learning curves rất quan trọng, nhất là khi dữ liệu âm thanh có nhiễu và các lớp dễ nhầm.

5.6 Lỗi 6. Không kiểm tra W&B

Chạy xong mà không có log trên W&B thì rất khó chứng minh quá trình thí nghiệm. Trước khi nộp bài, hãy mở dashboard kiểm tra lại run, config và metrics.

5.7 Lỗi 7. Kỳ vọng raw waveform chắc chắn thắng

Raw waveform nghe có vẻ “nguyên bản” hơn, nhưng không có nghĩa là luôn tốt hơn. Trong lab này, raw waveform là bài mở rộng để phân tích, không phải cuộc thi xem ai train mô hình to hơn.

6 Câu hỏi tự kiểm tra sau lab

Hãy tự trả lời ngắn gọn các câu hỏi sau:

1. Vì sao cần đưa audio về cùng sample rate?
2. Vì sao cần pad/crop audio về cùng độ dài?
3. MFCC là gì trong pipeline của bài lab này?
4. Input của MFCC + 1D-CNN có shape như thế nào?
5. Conv1D đang trượt theo chiều nào?
6. Vì sao cấu hình chính dùng MFCC thay vì raw waveform?
7. Dấu hiệu nào cho thấy mô hình bị overfitting?
8. Confusion matrix giúp phát hiện điều gì?
9. W&B giúp ích gì khi phải so sánh nhiều cấu hình?
10. Nếu raw waveform có kết quả thấp hơn MFCC, có thể giải thích như thế nào?

Nếu bạn trả lời rõ ràng được 10 câu này, nghĩa là bạn đã hiểu đúng tinh thần của lab.